Project Initialization and Planning Phase

Date	28 June 2026
Team ID	
Project Title	Rising Waters
Maximum Marks	5 Marks

Project Proposal (Proposed Solution) report
"The Rising Waters flood prediction system relies on a robust and highly scalable technology stack. The backend is powered by Python and Flask, seamlessly integrating with an advanced XGBoost machine learning classification model to rapidly process meteorological data. For secure and reliable data storage, the application utilizes a cloud-hosted PostgreSQL database (Neon.tech), managed through SQLAlchemy. The client-facing frontend employs responsive HTML5, CSS3, and JavaScript with Jinja2 templating to deliver an intuitive, glassmorphism-styled user interface. The entire system is version-controlled via GitHub and deployed online through Render to ensure continuous, high-availability access."

Project Overview	
Objective	The primary objective of the **Rising Waters** project is to develop a Machine Learning-based flood prediction system that analyzes weather and rainfall parameters to predict the possibility of flooding. The system aims to provide quick, reliable, and accurate predictions to help users take preventive actions before flood events occur.
Scope	The project focuses on predicting flood risk using historical weather and rainfall data. It includes data preprocessing, training a Random Forest machine learning model, developing a Flask-based web application, and deploying the application online to provide instant flood predictions through an easy-to-use interface.
Problem Statement	
Description	Floods cause significant damage to lives, agriculture, and infrastructure every year. Traditional warning methods are often delayed or rely heavily on manual monitoring. There is a need for an intelligent system capable of predicting flood risk quickly using weather and rainfall data.
Impact	The proposed solution enables early flood prediction, helping individuals, farmers, and disaster management authorities take precautionary measures. This reduces potential damage, improves disaster preparedness, and supports faster decision-making.
Proposed Solution	

Approach	The solution uses a **Random Forest Machine Learning algorithm** ,KNN,XG Boost trained on historical flood and weather data. The Flask web application accepts user inputs such as temperature, humidity, cloud cover, annual rainfall, seasonal rainfall, average June rainfall, and sub-rainfall values. The trained model processes these inputs and predicts whether there is a **Flood Chance** or **No Flood Chance**.
Key Features	Machine Learning-based flood prediction using Random Forest. User-friendly web interface developed using HTML, CSS, and Flask. Instant flood prediction based on weather parameters. Real-time prediction through a deployed web application. Responsive and easy-to-use interface. Online deployment using Render.
	- Real-time decision-making for quicker loan approvals. - Continuous learning to adapt to evolving financial landscapes.

Resource Requirement

Resource Type	**Description**	**Specification/Allocation**
Hardware		
Computing Resources	CPU/GPU specifications, number of cores	T4 GPU
Memory	RAM specifications	8 GB
Storage	Disk space for data, models, and logs	1 TB SSD
Software		
Frameworks	Python frameworks	Flask
Libraries	Additional libraries	scikit-learn, pandas, numpy, matplotlib, seaborn
Development Environment	IDE	Jupyter Notebook, pycharm
Data		
Data	Source, size, format	Kaggle dataset, 614, csv UCI dataset, 690, csv